

Experiment 5

Aim: To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Source Code:

```
-- =====  
-- 1. CREATE DEPARTMENT TABLE  
-- =====  
  
CREATE TABLE department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50) NOT NULL  
);  
  
-- Insert departments  
INSERT INTO department VALUES  
(1, 'IT'),  
(2, 'HR'),  
(3, 'Finance');  
  
-- =====  
-- 2. CREATE EMPLOYEE TABLE  
-- =====  
  
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL,  
    department_id INT,  
    manager_id INT NULL,  
    salary DECIMAL(10,2),  
    FOREIGN KEY (department_id)  
        REFERENCES department(department_id),  
    FOREIGN KEY (manager_id)  
        REFERENCES employee(employee_id)  
);  
  
-- Insert employees  
INSERT INTO employee VALUES  
(101, 'Amit', 1, NULL, 90000),  
(102, 'Neha', 1, 101, 65000),
```

```

(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

-- =====

-- 3. VIEW FOR DEPARTMENT SALARY SUMMARY

-- =====

CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY
    d.department_id,
    d.department_name;

-- Display the view

SELECT * FROM department_salary_summary;

-- =====

-- 4. VIEW FOR EMPLOYEE HIERARCHY

-- =====

CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary

```

```

FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;
-- Display the view
SELECT * FROM employee_hierarchy;

-- =====

-- 5. TEST VIEW UPDATABILITY

-- =====

-- Simple single-table view
CREATE VIEW employee_basic AS
SELECT
    employee_id,
    employee_name,
    salary
FROM employee;
-- Update salary through the view
UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;
-- Check updated record
SELECT *
FROM employee_basic
WHERE employee_id = 102;
-- Check original table also
SELECT *
FROM employee
WHERE employee_id = 102;

-- =====

-- 6. RECURSIVE CTE FOR REPORTING CHAINS

-- =====

WITH RECURSIVE reporting_chain AS (
    -- Anchor member:
    -- Employees who have no manager
    SELECT

```

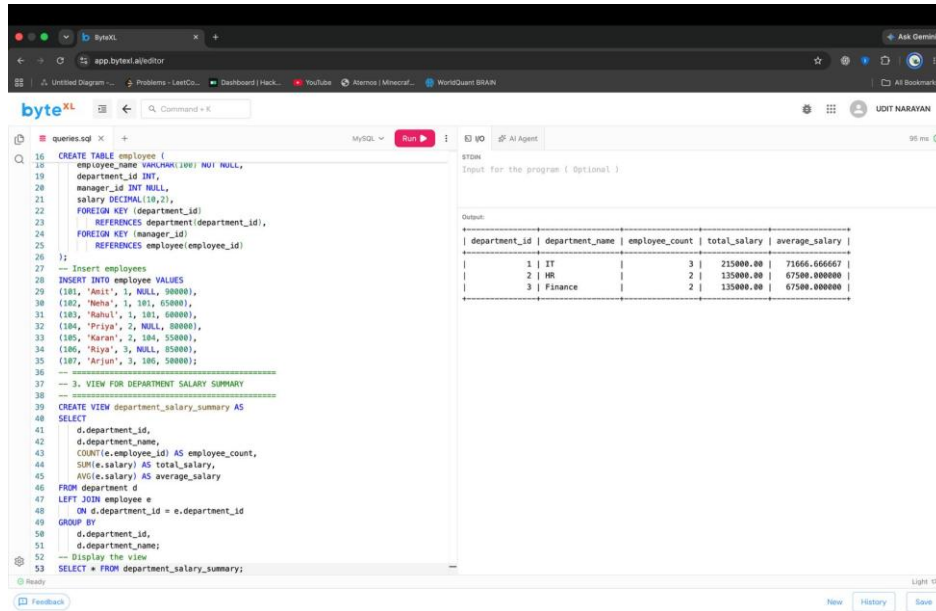
```

    employee_id,
    employee_name,
    manager_id,
    0 AS level,
    CAST(employee_name AS CHAR(1000)) AS reporting_path
FROM employee
WHERE manager_id IS NULL
UNION ALL
-- Recursive member:
-- Find employees reporting to each employee
SELECT
    e.employee_id,
    e.employee_name,
    e.manager_id,
    rc.level + 1,
    CONCAT(
        rc.reporting_path,
        ' -> ',
        e.employee_name
    )
FROM employee e
INNER JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    reporting_path
FROM reporting_chain
ORDER BY reporting_path;

```

OUTPUT:

1: DEPARTMENT SALARY:

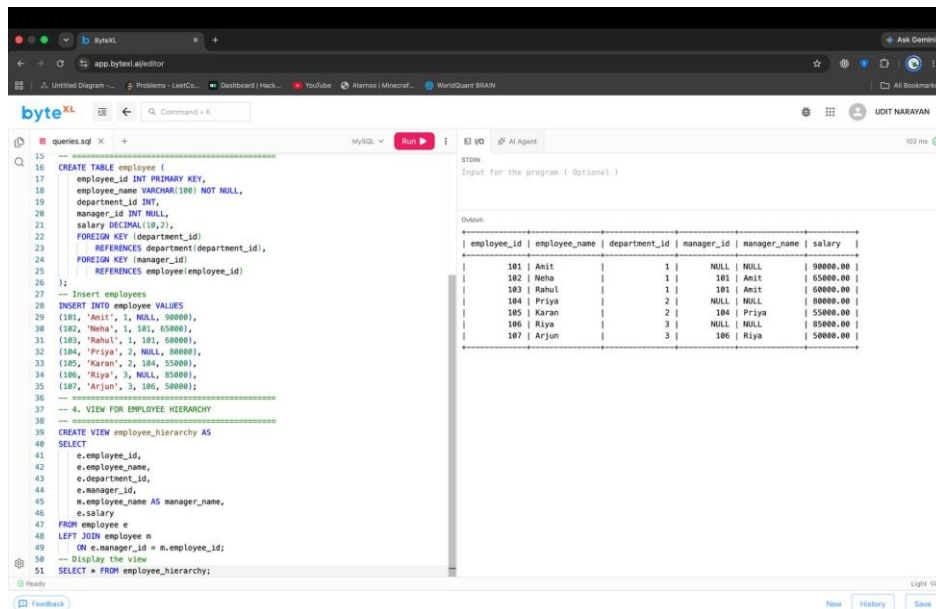


The screenshot shows the byteXL SQL editor interface. The SQL code defines an 'employee' table with columns for employee_name, department_id, manager_id, and salary. It includes foreign key constraints for department_id and manager_id. The code also inserts data for seven employees and creates a view named 'department_salary_summary' that aggregates data by department, showing employee count, total salary, and average salary.

```
16 CREATE TABLE employee (  
17   employee_name VARCHAR(100) NOT NULL,  
18   department_id INT,  
19   manager_id INT NULL,  
20   salary DECIMAL(10,2),  
21   FOREIGN KEY (department_id)  
22     REFERENCES department(department_id),  
23   FOREIGN KEY (manager_id)  
24     REFERENCES employee(employee_id)  
25 );  
26  
27 -- Insert employees  
28 INSERT INTO employee VALUES  
29   (101, 'Amit', 1, NULL, 90000),  
30   (102, 'Neha', 1, 101, 65000),  
31   (103, 'Rahul', 1, 101, 60000),  
32   (104, 'Priya', 2, NULL, 50000),  
33   (105, 'Karan', 2, 104, 55000),  
34   (106, 'Riya', 3, NULL, 85000),  
35   (107, 'Arjun', 3, 106, 50000);  
36  
37 -- 3. VIEW FOR DEPARTMENT SALARY SUMMARY  
38  
39 CREATE VIEW department_salary_summary AS  
40 SELECT  
41   d.department_id,  
42   d.department_name,  
43   COUNT(e.employee_id) AS employee_count,  
44   SUM(e.salary) AS total_salary,  
45   AVG(e.salary) AS average_salary  
46 FROM department d  
47 LEFT JOIN employee e  
48   ON d.department_id = e.department_id  
49 GROUP BY  
50   d.department_id,  
51   d.department_name;  
52  
53 -- Display the view  
54 SELECT * FROM department_salary_summary;
```

department_id	department_name	employee_count	total_salary	average_salary
1	IT	3	215000.00	71666.666667
2	HR	2	135000.00	67500.000000
3	Finance	2	135000.00	67500.000000

2: EMPLOYEE HEIRARCHY:

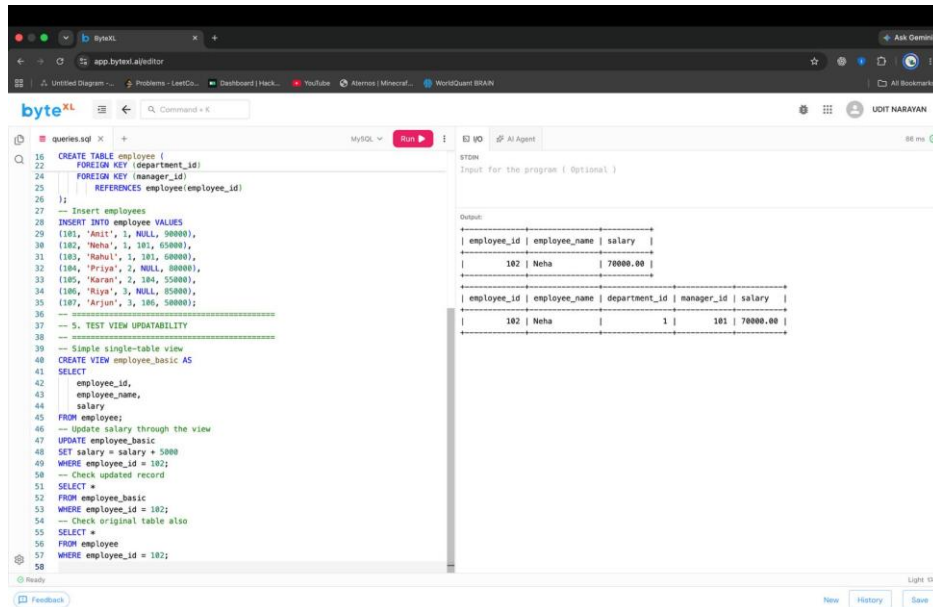


The screenshot shows the byteXL SQL editor interface. The SQL code defines an 'employee' table with columns for employee_id, employee_name, department_id, manager_id, and salary. It includes foreign key constraints for department_id and manager_id. The code also inserts data for seven employees and creates a view named 'employee_hierarchy' that displays the hierarchy of employees, showing their names, department IDs, manager IDs, and salaries.

```
16 CREATE TABLE employee (  
17   employee_id INT PRIMARY KEY,  
18   employee_name VARCHAR(100) NOT NULL,  
19   department_id INT,  
20   manager_id INT NULL,  
21   salary DECIMAL(10,2),  
22   FOREIGN KEY (department_id)  
23     REFERENCES department(department_id),  
24   FOREIGN KEY (manager_id)  
25     REFERENCES employee(employee_id)  
26 );  
27  
28 -- Insert employees  
29 INSERT INTO employee VALUES  
30   (101, 'Amit', 1, NULL, 90000),  
31   (102, 'Neha', 1, 101, 65000),  
32   (103, 'Rahul', 1, 101, 60000),  
33   (104, 'Priya', 2, NULL, 50000),  
34   (105, 'Karan', 2, 104, 55000),  
35   (106, 'Riya', 3, NULL, 85000),  
36   (107, 'Arjun', 3, 106, 50000);  
37  
38 -- 4. VIEW FOR EMPLOYEE HIERARCHY  
39  
40 CREATE VIEW employee_hierarchy AS  
41 SELECT  
42   e.employee_id,  
43   e.employee_name,  
44   e.department_id,  
45   m.employee_id AS manager_id,  
46   m.employee_name AS manager_name,  
47   e.salary  
48 FROM employee e  
49 LEFT JOIN employee m  
50   ON e.manager_id = m.employee_id;  
51  
52 -- Display the view  
53 SELECT * FROM employee_hierarchy;
```

employee_id	employee_name	department_id	manager_id	manager_name	salary
101	Amit	1	NULL	NULL	90000.00
102	Neha	1	101	Amit	65000.00
103	Rahul	1	101	Amit	60000.00
104	Priya	2	NULL	NULL	50000.00
105	Karan	2	104	Priya	55000.00
106	Riya	3	NULL	NULL	85000.00
107	Arjun	3	106	Riya	50000.00

3: UPDATABILITY TEST:



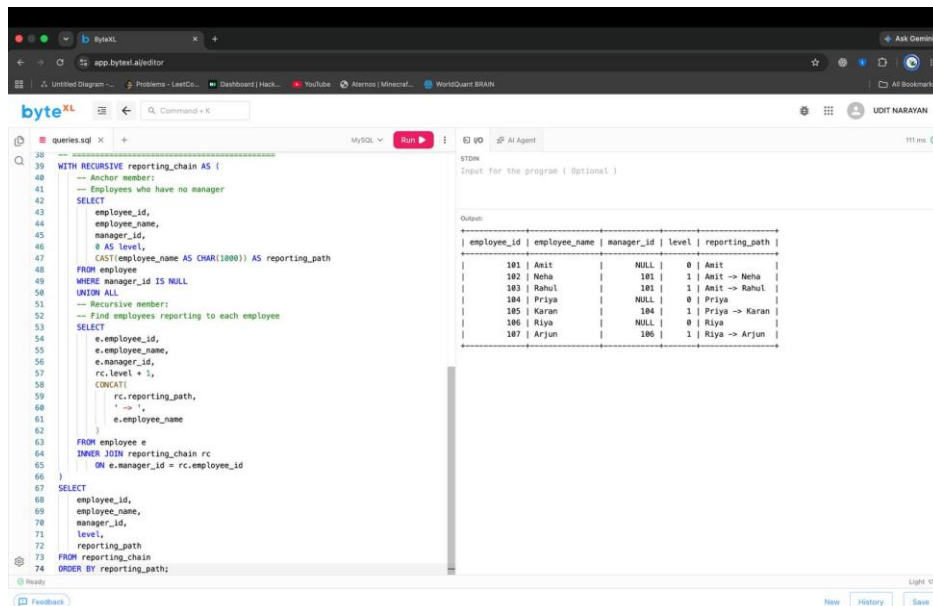
```
16 CREATE TABLE employee (
22   FOREIGN KEY (department_id)
24   FOREIGN KEY (manager_id)
25   REFERENCES employee(employee_id)
26 );
27 -- Insert employees
28 INSERT INTO employee VALUES
29 (101, 'Amit', 1, NULL, 90000),
30 (102, 'Neha', 1, 101, 60000),
31 (103, 'Rahul', 1, 101, 60000),
32 (104, 'Priya', 2, NULL, 80000),
33 (105, 'Karan', 2, 104, 55000),
34 (106, 'Riya', 3, NULL, 85000),
35 (107, 'Arjun', 3, 106, 50000);
36
37 -- 5. TEST VIEW UPDATABILITY
38 -- =====
39 -- Simple single-table view
40 CREATE VIEW employee_basic AS
41 SELECT
42   employee_id,
43   employee_name,
44   salary
45 FROM employee;
46 -- Update salary through the view
47 UPDATE employee_basic
48 SET salary = salary + 5000
49 WHERE employee_id = 102;
50 -- Check updated record
51 SELECT *
52 FROM employee_basic
53 WHERE employee_id = 102;
54 -- Check original table also
55 SELECT *
56 FROM employee
57 WHERE employee_id = 102;
58
```

Output:

employee_id	employee_name	salary
102	Neha	70000.00

employee_id	employee_name	department_id	manager_id	salary
102	Neha	1	101	70000.00

4: RECURSIVE CTE REPORTING CHAIN:



```
38 WITH RECURSIVE reporting_chain AS (
39   -- Anchor member:
40   -- Employees who have no manager
41   SELECT
42     employee_id,
43     employee_name,
44     manager_id,
45     0 AS level,
46     CAST(employee_name AS CHAR(1000)) AS reporting_path
47 FROM employee
48 WHERE manager_id IS NULL
49 UNION ALL
50 -- Recursive member:
51 -- Find employees reporting to each employee
52 SELECT
53   e.employee_id,
54   e.employee_name,
55   e.manager_id,
56   rc.level + 1,
57   CONCAT(
58     rc.reporting_path,
59     ' -> ',
60     e.employee_name
61   ) AS reporting_path
62 FROM employee e
63 INNER JOIN reporting_chain rc
64 ON e.manager_id = rc.employee_id
65
66 )
67 SELECT
68   employee_id,
69   employee_name,
70   manager_id,
71   level,
72   reporting_path
73 FROM reporting_chain
74 ORDER BY reporting_path;

```

Output:

employee_id	employee_name	manager_id	level	reporting_path
101	Amit	NULL	0	Amit
102	Neha	101	1	Amit -> Neha
103	Rahul	101	1	Amit -> Rahul
104	Priya	NULL	0	Priya
105	Karan	104	1	Priya -> Karan
106	Riya	NULL	0	Riya
107	Arjun	106	1	Riya -> Arjun